Computation Offloading and Resource Allocation in Unmanned Aerial Vehicle Networks

Binghong Liu, Chenxi Liu, *Senior Member, IEEE*, and Mugen Peng, *Fellow, IEEE*

***Abstract*—Deploying unmanned aerial vehicles (UAVs) as mobile edge computing (MEC) servers has been attracting significant attention, since the UAVs' inherent maneuverability and mobility can further reduce the distance between the users and computational functionalities. In the UAV-assisted MEC systems, the caching and offloading decision optimization, subject to the computation and energy constraints of the UAVs, is one of the key design issues. Against this backdrop, in this paper, we propose a novel cloud-edge framework to facilitate MEC in the UAV networks. Specifically, in our framework, the edge UAVs (EUAVs), together with the cloud, provide caching and computing services for the terrestrial users. In order to minimize the weighted sum cost of latency and energy consumption, we jointly optimize the caching and offloading decisions, the EUAV deployment, the radio and computation resource allocation, while simultaneously satisfying the UAVs' cache and computation capacity constraints, as well as the users' latency and energy consumption constraints. To solve this NP-hard problem, we propose a sequential convex programming (SCP) and sequential quadratic programming (SQP) based deep Q-learning (SS-DQN) algorithm. The proposed algorithm allows the system to adaptively adjust caching and offloading decisions with the EUAV deployment scheme and the resource allocation scheme obtained by SCP and SQP algorithms, respectively. Simulation results verify the superiority of our proposed algorithm compared to two benchmark schemes, and demonstrate how the combination of DRL and convex optimization can improve the system performance significantly.**

***Index Terms*—Unmanned aerial vehicles (UAVs), wireless caching, computation offloading.**

I. Introduction

THE demands for higher data rate and better network capability have experienced an exponential growth over the last few years. As forecasted in the Cisco report, by 2023, the number of networked devices will reach 29.3 billion, generating almost one zettabyte each year [1]. In order to meet users' needs and support a wide variety of fast-growing applications, which are generally delay-sensitive and computation-intensive, the mobile edge computing (MEC) [2], [3], [4] has been presented as a promising paradigm. Specifically, by bringing computing and cache resources closer to terminals, MEC can extend the cloud computing capability to the edge and as such reduce the transmission latency. Nevertheless, there are also many problems of the current edge computing system. For example, devices in internet-of-things (IoT) applications such as smart agriculture and forest fire monitoring are often far away from the core network [5], thus the rigidity of ground base stations (BSs) and their susceptibility to infrastructure failure may prevent them from satisfying the quality-of-service (QoS) requirements.

Recently, the UAVs have been increasingly applied in the edge computing system to open new opportunities for the next generation mobile network [6], [7], [8], [9], and the UAV-assisted MEC system has been attracting widespread research attention [10], [11], [12], [13], [14], due to the flexible and maneuverable potential of UAVs to deliver prompt services [15], [16], [17]. Compared with conventional MEC systems, the UAV-assisted MEC system can offer more flexible and on-demand services, where the UAVs act as the MEC servers to supply faster response to the user requests via high-quality line of sight (LoS) communication links [18]. Specifically, in [10], a UAV was deployed to facilitate the MEC service for the IoT devices, and the latency and energy consumption were minimized by jointly optimizing the task split ratio, UAV deployment and resource allocation, where the task split ratio is defined as the ratio of the offloaded bits to the total bits of the computation task. In [11], the scenario was expanded into a swarm of the UAVs, in which each UAV had a time-varying task to be completed. With the same goal as in [10], the transmission mode and the offloading decisions were jointly optimized. In [12], the throughput of the UAV-assisted MEC systems was maximized by jointly planning the trajectory and scheduling the tasks under constraints of transmission rate and task atomicity. In [13], the ultra-reliable low-latency computation offloading problem was studied in a UAV-assisted IoT network to maximize the transmission rate. In [14], the 5G network slice extension problem was investigated in a UAV-assisted edge computing system, where the system controller was able to turn on and off the computing functions of UAVs to serve users flexibly and save energy.

The aforementioned works [10], [11], [12], [13], [14] focused on single-tier UAV-assisted MEC networks. As such, the limited computation capacity and energy storage of the UAVs [17] becomes a vital issue that hinder the real implementations of the UAV-assisted MEC networks. One possible solution to this problem is to incorporate the cloud computing as a supplement to the edge UAVs (EUAVs). In such multi-tier cloud-edge net-

Manuscript received 15 October 2022; accepted 14 November 2022. Date of publication 17 November 2022; date of current version 18 April 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2900200, in part by the National Natural Science Foundation of China under Grants 62001047, 61925101, 61921003, and 61831002, in part by the Young Elite Scientist Sponsorship Program by China Institute of Communications, and in part by Beijing Municipal Science and Technology under Grant Z211100004421017. The review of this article was coordinated by Prof. Alfredo Luigi Grieco. *(Corresponding author: Chenxi Liu.)*

The authors are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: binghong_liu@bupt.edu.cn; chenxi.liu@bupt.edu.cn; pmg@bupt.edu.cn).

Digital Object Identifier 10.1109/TVT.2022.3222907

0018-9545 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

works, the cloud center has powerful computing resource and huge cache capacity, to which the EUAVs can connect via the fronthaul links, thus making up the deficiency of UAV-assisted MEC to achieve cost-effective computation with latency satisfaction. Despite its potential, little efforts has so far been devoted to the design and deployment of the UAV-assited cloud-edge computing systems [19], [20], [34]. The authors in [19] considered a typical scenario, where one UAV was dispatched as the MEC server to handle the computation tasks together with the cloud. The resource allocation and three-dimensional (3D) trajectory design were jointly optimized to minimize the energy consumption. Similar to [19], the authors in [34] studied the energy consumption minimization problem by jointly optimizing the computation resource scheduling, bandwidth allocation and UAV trajectory. In [20], a UAV-assisted cloud-edge system for virtual reality (VR) applications was considered, where the UAVs were deployed as aerial base stations (BSs) to cache, process and deliver VR contents from the cloud to users. The user association, caching policy and locations of UAVs were jointly optimized to minimize the latency.

A. Motivations and Contributions

Based on the above observations, although the UAV-assisted computing and caching have been separately discussed in previous studies [19], [20] and [34], few researches have comprehensively considered both of them. By avoiding the duplicate transmission of popular file contents and handling the burst traffic timely, the introduction of caching mechanism into computing can effectively improve the resource reuse, reduce the latency, and enhance the user experience. As such, how to exploit the relationship between caching and computing to schedule tasks between the edge and the cloud, and further, to balance the latency and energy consumption of the UAV-assisted cloud-edge system is worth to be studied. The reason is that the uneven user requests and heterogeneity of the cloud-edge architecture pose a challenge to the selection of cached data and offloading location. In addition, the aforementioned works mainly considered the use of either convex optimization method or the deep reinforcement learning (DRL) method. For example, works [10], [12], [13], [19], [20], and [34] adopted the traditional convex optimization method, while works [11] and [14] utilized the DRL method. However, the combination of these two methods can contribute to a more efficient system. Therefore, in this paper, we exploit the caching and computing functions of the edge server and consider a UAV-assisted cloud-edge system, in which the EUAVs, together with the cloud, provide caching and computation offloading services for the terrestrial users. We formulate the cost minimization problem and reveal its dependence on caching and offloading decisions, resource allocation, as well as UAV deployment strategies. The contributions of this paper are summarized as follows.

- We formulate a cost minimization problem for the considered multi-tier UAV-assited cloud-edge framwork, in which the balance between the latency and energy consumption can be achieved by jointly optimizing the caching and offloading decisions, the power and computation resource allocations, as well as the UAVs deployment, subject to the constraints on the computation and caching capacity of the EUAVs, and the latency and energy consumption of the users.
- We show how the formulated problem can be decomposed to three subproblems, namley, the EUAV deployment subproblem, transmit power and computation capacity allocation subproblem, and caching and offloading decision subproblem. Sequential convex programming (SCP), sequential quadratic programming (SQP), and deep Q-learning (DQN) algorithms are applied to solve these subproblems, respectively. Note that the proposed SCP and SQP-based DQN (SS-DQN) algorithm is a hybrid method that combines the benefits of both the convex optimization method and the DRL method. Specifically, the convex optimization is utilized to accelerate the convergence rate of the DRL by reducing the large state-action space.
- We exam the impacts of key parameters, i.e. cache and computation capacities, task sizes, on the cost performance, demonstrating the superiority of our proposed algorithm over various benchmark solutions. We further show that our proposed algorithm works well under the stringent latency constraints of the users, indicating the applicability of our proposed algorithm in practical UAV-assisted cloud-edge systems.

B. Organization

The rest of this paper is organized as follows. Section II presents the system model of the considered UAV-assisted cloud-edge system, analyzes the latency and energy consumption for executing tasks, and formulates the weighted sum cost minimization problem. Section III presents the proposed algorithm that solve the formulated problem. Numerical results and related discussions are provided in Section IV. Finally, Section V draws the conclusion. Unless otherwise specified, the notations used throughout the paper are summarized in Table I.

II. System Model and Problem Formulation

In this paper, we consider a UAV-assisted cloud-edge network, which consists of a cloud layer, an edge layer, and a terminal layer. In the edge layer, the UAVs with limited cache and computing capacities are deployed as the MEC servers in the vicinity of users. In the cloud layer, a tethered UAV (TUAV) physically wired to a ground control center (GCC) of power caching and computing capabilities is deployed. Both the energy and data transmission between the TUAV and the GCC are aggregated in the same tether [35], which not only enables the TUAV to maintain longer flight time, but also ensures the reliable communication at higher data rate. While due to the weight of the fiber, the tether length is usually limited to a range between 80 m and 150 m [36]. As such, we assume that the transmission latency between the TUAV and GCC is negligible, due to the use of high-speed fiber link and the limited tether length, and for the sake of simplicity, use the TUAV to represent the GCC [37]. The set of EUAVs and ground users are denoted by $\mathcal{N} = \{1, 2, \ldots, N\}$ and $\mathcal{M} = \{1, 2, \ldots, M\}$, respectively. The two-dimensional coordinate of EUAV n, TUAV and user m are $\mathbf{u}_n = [x_n, y_n]$, $\mathbf{u}_0 = [x_0, y_0]$, and $\mathbf{u}_m = [x_m, y_m]$, respectively.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LIST OF NOTATIONS

Notation	Description	Notation	Description
N	Number of EUAVs	ϕ_n,ψ_n	Cache and computation capacity of EUAV n
M	Number of users	f_m,$f_{n,m}$,$f_{c,m}$	CPU frequency of user m, EUAV n and cloud
h_n	Flight height of EUAV n	$\alpha_{n,m}$	Caching decision variable
h_0	Flight height of TUAV	$\beta^E_{m,n}$,$\beta^C_{m,n}$	Offloading decision variables
$h^{AtG}_{m,n}$	Air-to-ground channel coefficients	$q_{n,m}$	Transmit power of EUAV n to user m
$\hat{h}_{m,n}$	Small scale fading	$p_{m,n}$	Transmit power of user m to EUAV n
$L_{m,n}$	Large scale fading	p_{cloud}	Transmit power of TUAV
h^{AtA}_n	Air-to-air channel coefficients	p_{uav}	Transmit power EUAVs to TUAV
$R^{E,Down}_{n,m}$, $R^{E,Up}_{m,n}$	Maximum achievable rate of EUAV-user link	C_m,O_m	Size of the cache file and computation task
$r^{E,Down}_{n,m}$, $r^{E,Up}_{m,n}$	Outage-aware achievable rate of EUAV-user link	F_m	Required CPU cycles
$R^{C,Down}_{n,m}$, $R^{C,Up}_{m,n}$	Transmission rate of TUAV-EUAV link	γ	Learning rate
$B_{m,n}$	Bandwidth between user m and EUAV n	$\mathcal{E}$	Experience replay buffer
B_n	Bandwidth between TUAV and EUAV n	N_b	Batch size

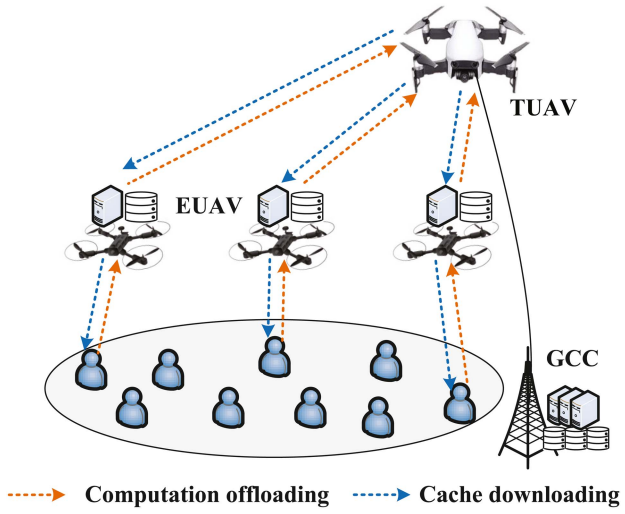


Fig. 1. Illustration of the UAV-assisted cloud-edge system.

The height of EUAV n and TUAV are fixed as h_n and h_0, respectively.

As shown in Fig. 1, the transmission can be divided into two phases, namely, the caching content delivery phase and the computation offloading phase. In the caching content delivery phase, the users obtain the requested files from the the cache servers (i.e., the EUAVs or the TUAV). In the computation offloading phase, the users decide where to complete the computation tasks based on the obtained files in the caching content delivery phase. Note that we assume that the computation can only be performed after the files are obtained. This assumptions is practical in many scenarios. For the real-time gaming applications, the game files with higher request rate need to be cached closer to users to reduce the computational burden of the cloud and lower the latency [24], and the user equipments perform subsequent computations based on these files.

A. The Communication Model

We assume that the EUAVs can communicate with both the TUAV and the terrestrial users, while the terrestrial users can only communicate with the EUAVs, due to the long distance between the TUAV and the users. In order to efficiently control the interference, the users and UAVs occupy orthogonal bandwidth for transmission. The air-to-ground communication link between user m and EUAV n is modeled as follows

$$h^{AtG}_{m,n} = \hat{h}_{m,n} L_{m,n}, \tag{1}$$

where $\hat{h}_{m,n}$ and $L_{m,n}$ represent the small-scale and large-scale fading, respectively. Specifically, $\hat{h}_{m,n}$ is assumed to follow the Rician fading [21], given by

$$\hat{h}_{m,n} = \left(\sqrt{\frac{Q_{m,n}}{Q_{m,n}+1}} \bar{h}_{m,n} + \sqrt{\frac{1}{Q_{m,n}+1}} \tilde{h}_{m,n} \right), \tag{2}$$

$\bar{h}_{m,n}$ is the deterministic LoS channel component, and $|\bar{h}_{m,n}|=1$ $\tilde{h}_{m,n}$ is the random scattering component, which is a circularly symmetric complex Gaussian (CSCG) random variable with zero mean and unit variance, namely, $\tilde{h}_{m,n} \sim \mathcal{CN}(0,1)$. Here CSCG means $\tilde{h}_{m,n}$ has independently and identically distributed real and imaginary parts, each follows $\mathcal{N}(0,1/2)$ [38] [39]. $Q_{m,n}$ denotes the Rician factor, $Q_{m,n} = A_1 \exp(A_2 \theta_{m,n})$, where A_1 and A_2 are environment-dependent constants, and $\theta_{m,n}$ represents the elevation angle, $\theta_{m,n} = \arcsin(\frac{h_n}{d_{n,m}})$. In (1), $L_{m,n}$ can be expressed as

$$L_{m,n} = \sqrt{\alpha_0 (d_{m,n})^{-a_1}}, \tag{3}$$

where α_0 is the power gain at reference distance $d_0 = 1$ m, $d_{m,n} = \sqrt{\|\mathbf{u}_n - \mathbf{u}_m\|^2 + h_n^2}$ is the distance between user m and EUAV n, and a_1 is the pathloss exponent of the air-to-ground link.

Based on (1)–(3), the downlink transmission rate of EUAV n for transmitting the requested cache file of user m is given by

$$R^{E,Down}_{n,m} = B_{m,n} \log_2 \left(1 + \frac{q_{n,m} \left\| h^{AtG}_{m,n} \right\|^2}{\sigma^2_{m,n}} \right), \tag{4}$$

where $\sigma^2_{m,n} = N_0 B_{m,n}$, $B_{m,n}$ is the transmission bandwidth between user m and EUAV n, N_0 is the noise power spectrum density. $q_{n,m}$ is the transmit power of EUAV n to user m.

Similarly, after receiving the requested data, the uplink transmission rate of user m for offloading its computation task to EUAV n is given by

$$R^{E,Up}_{m,n} = B_{m,n} \log_2 \left(1 + \frac{p_{m,n} \left\| h^{AtG}_{m,n} \right\|^2}{\sigma^2_{m,n}} \right), \tag{5}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

where $p_{m,n}$ is the transmit power of user m to EUAV n.

Based on (4) and (5), the downlink latency for transmitting requested cache file and the uplink latency for offloading computing task between EUAV n and user m can be expressed as $\bar{L}_{n,m} = \frac{O_m}{R^{E,Down}_{n,m}}$ and $\bar{L}_{m,n} = \frac{O_m}{R^{E,Up}_{m,n}}$, respectively. Due to the dynamic feature of the air-to-ground links, there exists a timeout probability that user m cannot successfully receive the requested cache files from the EUAV n. Mathematically, the timeout probability is given by

$$\begin{aligned} P^{out}_{n,m} &= P\left(L_{n,m} = \frac{O_m}{r^{E,Down}_{n,m}} < \bar{L}_{n,m}\right) \\ &= P\left(\left|\hat{h}_{m,n}\right|^2 < \frac{\sigma^2_{m,n}\left(2^{r^{E,Down}_{n,m}/B_{m,n}} - 1\right)}{\alpha_0 d^{-a_1}_{m,n} q_{n,m}}\right) \\ &= \Gamma_{n,m}\left(\frac{\sigma^2_{m,n}\left(2^{r^{E,Down}_{n,m}/B_{m,n}} - 1\right)}{\alpha_0 d^{-a_1}_{m,n} q_{n,m}}\right), \end{aligned} \tag{6}$$

where $\Gamma_{n,m}(\cdot)$ is the cumulative distribution function (CDF) of the random variable $|\hat{h}_{m,n}|^2$.

Denote δ as the maximum tolerable outage probability of the air-to-ground links. In order to ensure the reliable communication between the EUAVs and the users, $r^{E,Down}_{n,m}$ is chosen to satisfy the condition: $P^{out}_{n,m} \le \delta$ and we can obtain the outage-aware achievable rate as

$$r^{E,Down}_{n,m} = B_{m,n}\log_2\left(1 + \frac{\gamma^{down}_{n,m}\chi_{n,m}}{d^{a_1}_{m,n}}\right), \tag{7}$$

where $\chi_{n,m}$ is the unique solution of $\Gamma_{n,m}(\cdot) = \delta$, and $\gamma^{down}_{n,m} = \frac{q_{n,m}\alpha_0}{\sigma^2_{m,n}}$. To facilitate the analysis, we approximate $\chi_{n,m}$ by the log function [22], as such, $r^{E,Down}_{n,m}$ can be further expressed as

$$r^{E,Down}_{n,m} \doteq B_{m,n}\log_2\left(1+\left(C_1+\frac{C_2}{1+e^{-\left(C_3+C_4\frac{H_n}{d_{m,n}}\right)}}\right)\frac{\gamma^{down}_{n,m}}{d^{a_1}_{m,n}}\right), \tag{8}$$

where $C_1 > 0, C_2 > 0$ and $C_1 + C_2 = 1, C_3 < 0$ represents the positive logistic mid-point, and $C_4 > 0$ represents the logistic growth rate.

Similarly, the outage-aware achievable rate in the uplink from user m to EUAV n can be expressed as

$$r^{E,Up}_{m,n} \doteq B_{m,n}\log_2\left(1+\left(C_1+\frac{C_2}{1+e^{-\left(C_3+C_4\frac{H_n}{d_{m,n}}\right)}}\right)\frac{\gamma^{up}_{m,n}}{d^{a_1}_{m,n}}\right). \tag{9}$$

where $\gamma^{up}_{m,n} = \frac{p_{m,n}\alpha_0}{\sigma^2_{m,n}}$.

The air-to-air communication link from the EUAV to the TUAV is modeled as the quasi-static block fading LoS link, which is given by

$$h^{AtA}_n = \sqrt{\alpha_0 d^{-a_2}_n}, \tag{10}$$

where $d_n = \sqrt{\|\mathbf{u}_n - \mathbf{u}_0\|^2 + (h_n - h_0)^2}$ is the distance between EUAV n and TUAV, a_2 is the pathloss exponent of the air-to-air link.

Therefore, the downlink transmission rate of TUAV delivering user m's requested cache file to EUAV n is given by

$$R^{C,Down}_{n,m} = B_n\log_2\left(1 + \frac{p_{cloud}\left\|h^{AtA}_n\right\|^2}{\sigma^2_n}\right), \tag{11}$$

where $\sigma^2_n = N_0 B_n$, B_n is the transmission bandwidth between TUAV and EUAV n, p_{cloud} is the downlink transmit power of TUAV.

Similarly, the uplink transmission rate of EUAV n offloading user m's computation task to TUAV is given by

$$R^{C,Up}_{m,n} = B_n\log_2\left(1 + \frac{p^{UAV}_n\left\|h^{AtA}_n\right\|^2}{\sigma^2_n}\right), \tag{12}$$

where p^{UAV}_n is the transmit power of EUAV n for offloading user m's computing task to TUAV. For the sake of simplicity, we assume that the transmit power of all the EUAVs is fixed as p_{uav}.

B. Caching and Computing Model

The task of user m can be represented as $S_m = (C_m, O_m, F_m)$, in which C_m and O_m are the size of the requested cache file and the related computation task, respectively, and F_m is the required CPU cycles for computing the task. Note that each EUAV is able to provide its cached data for users who request to compute the data. The un-cached files need to be firstly obtained from the cloud through backhaul, and then transmitted to users. We denote $\alpha_{n,m}$ as the caching decision variable that indicates whether the file requested by user m is cached in EUAV n. Specifically, $\alpha_{n,m} = 1$ means that the file is cached, otherwise, $\alpha_{n,m} = 0$.

Since the computation capabilities of the terrestrial users are in general limited, we consider that the users can offload their tasks to EUAVs or even the cloud via the EUAVs. Assuming that the computation task has to be offloaded as a whole, we introduce the offloading decision variable $\beta^E_{m,n} \in \{0,1\}$ and $\beta^C_{m,n} \in \{0,1\}$ to indicate where to offload the tasks. Specifically, $\beta^E_{m,n} = 1$ means that user m choose to offload its computation task to EUAV n, and $\beta^C_{m,n} = 1$ means that the task is offloaded to the cloud. Note that in this paper, the binary offloading can be transformed to the partial offloading when the packet segmentation is fine enough. Specifically, when we divide the task, which is previously offloaded as a whole in the binary offloading mode, into L equal-size segments, the offloading becomes the partial offloading in another sense, and L represents the segmentation index.

In the following, we analyze the latency and energy consumption of our considered network in detail.

1) Latency: Note that the users firstly need to obtain the requested cache files from the EUAVs or the cloud. Based on the obtained cache data, the users start the computation. There are three modes that the users that can choose from for computing the tasks, namely, the local computing mode, the edge computing mode, and the cloud computing mode. We assume that the transmission of cached files and the offloading of computation tasks are carried out by sequence and independently with each other. Then, the latency for transmitting the requested file to user

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

m from EUAV n is given by

$$L_{m,n}^{cache} = (1-\alpha_{n,m})\frac{C_m}{R_{n,m}^{C,Down}} + \frac{C_m}{r_{n,m}^{E,Down}}. \quad (13)$$

In the local computing mode, the latency consists of the transmission latency for delivering the cache file and the computation latency for processing tasks locally, which is given by

$$L_{m,n}^{local} = L_{m,n}^{cache} + \frac{F_m}{f_m}, \quad (14)$$

where f_m represents the local processing ability of user m.

In the edge computing mode, the latency consists of the transmission latency for delivering the cache file, the transmission latency for offloading user m's task to EUAV n, and the edge computing latency, which is given by

$$L_{m,n}^{edge} = L_{m,n}^{cache} + \frac{O_m}{r_{m,n}^{E,Up}} + \frac{F_m}{f_{n,m}}, \quad (15)$$

where $f_{n,m}$ represents the allocated edge computing resources of EUAV n to process the task offloaded by user m.

In the cloud computing mode, the latency consists of the transmission latency for delivering the cache file, the transmission latency for offloading user m's task to EUAV n, and then to TUAV, and the cloud computing latency, which is given by

$$L_{m,n}^{cloud} = L_{m,n}^{cache} + \frac{O_m}{r_{m,n}^{E,Up}} + \frac{O_m}{R_{m,n}^{C,Up}} + \frac{F_m}{f_{c,m}}, \quad (16)$$

where $f_{c,m}$ represents the allocated computing resources of the cloud center to process the task offloaded by user m.

In this paper, we mainly focus on the applications in which the amount of uplink data is significantly larger than that of downlink data. Therefore, as in the previous studies [10], [11], [13], [19], [20], the latency for receiving the outcome of computation tasks is ignored. As such, the total latency for executing tasks of user m is given by

$$L_{m,n}^{total} = \left(1-\beta_{m,n}^{E}-\beta_{m,n}^{C}\right) L_{m,n}^{local} + \beta_{m,n}^{E} L_{m,n}^{edge} + \beta_{m,n}^{C} L_{m,n}^{cloud}. \quad (17)$$

2) Energy Consumption: Next, we focus on the energy consumption of our considered framework. The energy consumption for transmitting the requested cache file from EUAV n to user m, is given by

$$E_{m,n}^{cache} = (1-\alpha_{m,n})\frac{C_m}{R_{n,m}^{C,Down}} \cdot p_{cloud} + \frac{C_m}{r_{n,m}^{E,Down}} \cdot q_{n,m}. \quad (18)$$

Therefore, the energy consumption of the local computing mode is given by

$$E_{m,n}^{local} = E_{m,n}^{cache} + \varepsilon f_m^2 F_m. \quad (19)$$

where ε is the effective switched capacitance determined by the specific chip architecture.

The energy consumption of the edge computing mode is given by

$$E_{m,n}^{edge} = E_{m,n}^{cache} + \frac{O_m}{r_{m,n}^{E,Up}} \cdot p_{m,n} + E_{n,m}^{com}. \quad (20)$$

Note that $E_{n,m}^{com}$ is the energy consumption of EUAV n for computing the offloaded task of user m, $E_{n,m}^{com} = \frac{\varepsilon F_m}{f_{n,m}}(e_1(f_{n,m})^v + e_2)$, where $e_1 > 0$ and $e_2 > 0$ are constants obtained by the offline power fitting, and $2.5 < v < 3$ according to [26].

The energy consumption of the cloud computing mode is given by

$$E_{m,n}^{cloud} = E_{m,n}^{cache} + \frac{O_m}{r_{m,n}^{E,Up}} \cdot p_{m,n} + \frac{O_m}{R_{m,n}^{C,Up}} \cdot p_{uav} + E_{c,m}^{com}. \quad (21)$$

Similarly, $E_{c,m}^{com}$ is the energy consumption of the cloud center for computing the offloaded task of user m, $E_{c,m}^{com} = \frac{\varepsilon F_m}{f_{c,m}}(e_3(f_{c,m})^v + e_4)$.

Then, the total energy consumption for executing tasks of user m is given by

$$E_{m,n}^{total} = \left(1-\beta_{m,n}^{E}-\beta_{m,n}^{C}\right) E_{m,n}^{local} + \beta_{m,n}^{E} E_{m,n}^{edge} + \beta_{m,n}^{C} E_{m,n}^{cloud}. \quad (22)$$

Note that the energy consumption of each EUAV n consists of the energy consumed by data transmission, task computation and hovering. As referred in [40], [41], the energy consumed by hovering is much smaller than that consumed by propulsion [42]. Moreover, the energy consumed by data transmission is much smaller than that consumed by computing and hovering [43]. In this paper, we only consider the UAVs' hovering situation in a short-time period. As such, the energy consumption of each EUAV is within its energy capacity.

Based on the above analysis on latency and energy consumption, respectively, the entire cost of the UAV-assisted cloud-edge system is defined as follows

$$\begin{aligned} W^{total} &= \sum_{n=1}^{N}\sum_{m=1}^{M} W_{m,n}^{total} \\ &= \sum_{n=1}^{N}\sum_{m=1}^{M} \left(\omega_{m,1} L_{m,n}^{total} + \omega_{m,2} E_{m,n}^{total}\right), \end{aligned} \quad (23)$$

where $\omega_{m,1}$ and $\omega_{m,2}$ are the weight coefficients for balancing the latency and the energy consumption. Note that the users with different service types have various preference for latency (represented as QoS index 1) and energy consumption (represented as QoS index 2). Assuming that each QoS index can be further divided into K preference levels, in which a higher level means that this index is more favored [25] and has more significant effect on the users. The preference level of user m on QoS index $i, (i = 1, 2)$, can be represented as $k_m^i \in \{1, 2, \ldots, K\}$. In addition, we introduce $a_m^{i,j} = k_m^i / k_m^j$ to measure user m's relative preference degree on the QoS index 1 compared with the QoS index 2. Therefore, the weight coefficient can be further represented as follows

$$\omega_{m,i} = \frac{1}{2}\sum_{j=1}^{2} \frac{a_m^{i,j}}{a_m^{1,j} + a_m^{2,j}}, \forall i = \{1, 2\}. \quad (24)$$

C. Problem Formulation

In this paper, the goal of our work is to minimize the total cost of the UAV-assisted cloud-edge system by jointly optimizing the caching and offloading decision schemes $\boldsymbol{\alpha} = \{\alpha_{n,m} | \forall m, \forall n\}$ and $\boldsymbol{\beta} = \{\beta_{m,n}^{E}, \beta_{m,n}^{C} | \forall m, \forall n\}$, the EUAV deployment[1] scheme $\mathbf{u} = \{\mathbf{u}_n | \forall n\}$, the power allocation scheme

[1] In this paper, we assume that the TUAV is located right above the GCC, due to the limited mobility introduced by the tether.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

$\mathbf{P} = \{p_{m,n}, q_{n,m} | \forall m, \forall n\}$, and the computing capacity allocation scheme $\mathbf{F} = \{f_{n,m}, f_{c,m} | \forall m, \forall n\}$, while simultaneously satisfying the cache and computing capacity constraints of UAVs, as well as the latency and energy consumption constrains of users' tasks. Mathematically, the optimization problem can be expressed as

$$
\begin{aligned}
&\min_{\boldsymbol{\alpha},\boldsymbol{\beta},\mathbf{u},\mathbf{P},\mathbf{F}} \quad W^{total} \\
&\text{s.t.} \sum_{n=1}^{N} \left(\beta_{m,n}^{E} + \beta_{m,n}^{C}\right) = 1, (a) \\
&\beta_{m,n}^{E}, \beta_{m,n}^{C}, \alpha_{n,m} \in \{0,1\} (b) \\
&\sum_{m=1}^{M} \beta_{m,n}^{E} f_{n,m} \leq \psi_n, (c) \\
&\sum_{m=1}^{M} \alpha_{n,m} C_m \leq \varphi_n, (d) \\
&L_{m,n}^{total} \leq T_{\max}, (e) \\
&E_{m,n}^{total} \leq E_{\max}, (f) \\
&0 \leq p_{m,n} \leq p_m^{\max}, (g) \\
&0 \leq q_{n,m} \leq q_n^{\max}, (h) \\
&x_{\min} \leq x_n \leq x_{\max}, y_{\min} \leq y_n \leq y_{\max}, (i)
\end{aligned} \tag{25}
$$

where constraint (25a) means that user m can only choose to compute the task O_m locally or offload it to the edge or the cloud for further computation, constraint (25b) regulates that the caching and offloading decision variables are binary variables, constraint (25c) means that the computation tasks offloaded to EUAV n should not exceed its computing capacity ψ_n, constraint (25d) means that the content files cached in EUAV n should not exceed its cache capacity φ_n, constraints (25e) and (25f) mean that the time and energy consumption for completing tasks of user m shouldn't exceed the corresponding threshold, which are application-specific and changes with user needs, constraints (25g) and (25h) are the transmit power constraints for the users and EUAVs, respectively, and constraint (25i) restricts the scope of the EUAVs' deployment area.

Note that (25) is a mixed integer non-linear programming (MINLP) problem, due to the discrete and continuous variables, as well as the non-convex objective function and constraints. As such, (25) is in general difficult to solve. To address this issue, in the next section, taking the combinatorial characteristic and inherent non-convexity of (25) into consideration, we propose a hybrid problem-solving framework that minimizes the weighted sum cost of latency and energy consumption of our considered networks, while inducing relatively low computational complexity.

III. Proposed Hybrid Problem-Solving Framework: The SS-DQN Algorithm

In this section, we propose a novel SS-DQN algorithm to solve the optimization problem in (25). Specifically, we decompose (25) into three subproblems, namely, the EUAV deployment subproblem, the resource allocation subproblem, as well as the caching and offloading decision subproblem. Then, the SCP-, SQP-, and DQN-based algorithms are proposed to solve these three subproblems, respectively. The proposed SCP- and SQP-based algorithms optimize the deployment and the resource allocation of the EUAVs, respectively. Based on which, the proposed DQN-based algorithm allows for adaptively adjusting the caching and offloading decisions of the system such that the system cost can be minimized.

Algorithm 1: SCP-based EUAV deployment algorithm.

Input : resource allocation schemes $\mathbf{F}, \mathbf{P}$, caching and offloading decision schemes $\boldsymbol{\alpha}, \boldsymbol{\beta}$, maximum number of iterations I_{iter}^{1}, error tolerance ε_1.

Output: the optimal EUAV deployment scheme $\mathbf{u}^*$.

```
Randomly initialize u(0);
i = 1;
repeat
    Solve the convex optimization problem in (43) to get
      the deployment scheme u(i);
    Calculate W^total(i) based on u(i);
    Set i = i + 1;
until |W^total(i) − W^total(i − 1)| < ε1 or i > I^1_iter;
```

A. EUAV Deployment Subproblem

We first focus on the EUAV deployment subproblem. With the given caching and offloading decision scheme and the resource allocation scheme, the problem in (25) can be expressed as

$$
\begin{aligned}
&\min_{\mathbf{u}} \quad W^{total} \\
&\text{s.t.} \quad (25e)(25f)(25i).
\end{aligned} \tag{26}
$$

Note that the objective function is non-convex. To address this issue, we transform W^{total} to a convex form following the steps. According to (8)–(9) and (13)–(23), we first express $W_{m,n}^{total}$ as

$$
\begin{aligned}
W_{m,n}^{total} = \delta_{m,n} &+ \frac{O_m}{r_{m,n}^{E,Up}} \left(\omega_{m,1} + \omega_{m,2} p_{m,n}\right) \left(\beta_{m,n}^{E} + \beta_{m,n}^{C}\right) \\
&+ \frac{O_m \beta_{m,n}^{C}}{R_{m,n}^{C,Up}} \left[\left(\omega_{m,1} + \omega_{m,2} p_{uav}\right)\right] \\
&+ \frac{C_m}{r_{n,m}^{E,Down}} \left(\omega_{m,1} + \omega_{m,2} q_{n,m}\right) \\
&+ \frac{C_m \left(1 - \alpha_{n,m}\right)}{R_{n,m}^{C,Down}} \left(\omega_{m,1} + \omega_{m,2} p_{cloud}\right),
\end{aligned} \tag{27}
$$

where $\delta_{m,n}$ is expressed as

$$
\begin{aligned}
\delta_{m,n} = &\left[\left(1 - \beta_{m,n}^{E} - \beta_{m,n}^{C}\right) \left(\omega_{m,1} \frac{F_m}{f_m} + \omega_{m,2} \varepsilon f_m^2 F_m\right)\right. \\
&+ \beta_{m,n}^{E} \left(\left(\omega_{m,1} + \omega_{m,2} e_2\right) \frac{F_m}{f_{n,m}} + \omega_{m,2} e_1 f_{n,m}^{v-1} F_m\right) \\
&\left.+ \beta_{m,n}^{C} \left(\left(\omega_{m,1} + \omega_{m,2} e_4\right) \frac{F_m}{f_{c,m}} + \omega_{m,2} e_3 f_{c,m}^{v-1} F_m\right)\right].
\end{aligned} \tag{28}
$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

By denoting

$$\begin{aligned} c_1 &= \left(\omega_{m,1}+\omega_{m,2}p_{m,n}\right)\left(\beta^E_{m,n}+\beta^C_{m,n}\right), \\ c_2 &= \beta^C_{m,n}\left(\omega_{m,1}+\omega_{m,2}p_{uav}\right), \\ c_3 &= \left(\omega_{m,1}+\omega_{m,2}q_{n,m}\right), \\ c_4 &= \left(1-\alpha_{n,m}\right)\left(\omega_{m,1}+\omega_{m,2}p_{cloud}\right), \end{aligned} \tag{29}$$

we can re-express (26) as

$$\min_{\mathbf{u},\mathbf{e},\mathbf{g}} \sum_{n=1}^{N}\sum_{m=1}^{M}\left(\delta_{m,n}+\frac{c_1 O_m}{e^{E,Up}_{m,n}}+\frac{c_2 O_m}{e^{C,Up}_{m,n}}+\frac{c_3 C_m}{e^{E,Down}_{n,m}}+\frac{c_4 C_m}{e^{C,Down}_{n,m}}\right)$$

$$\text{s.t.} \quad (25i)$$

$$L^{total}_{m,n}\left(e^{E,Up}_{m,n}, e^{C,Up}_{m,n}, e^{E,Down}_{n,m}, e^{C,Down}_{n,m}\right) \le T_{\max}, (a)$$

$$E^{total}_{m,n}\left(e^{E,Up}_{m,n}, e^{C,Up}_{m,n}, e^{E,Down}_{n,m}, e^{C,Down}_{n,m}\right) \le E_{\max}, (b)$$

$$e^{E,Up}_{m,n} \le r^{E,Up}_{m,n}\left(g_{m,n}, \mathbf{u}_n\right), (c)$$

$$e^{E,Down}_{n,m} \le r^{E,Down}_{n,m}\left(g_{m,n}, \mathbf{u}_n\right), (d)$$

$$g_{m,n} \le \frac{h_n}{d_{m,n}\left(\mathbf{u}_n\right)}, (e)$$

$$e^{C,Up}_{m,n} \le R^{C,Up}_{m,n}\left(\mathbf{u}_n\right), (f)$$

$$e^{C,Down}_{n,m} \le R^{C,Down}_{n,m}\left(\mathbf{u}_n\right). (g) \tag{30}$$

where $e^{E,Up}_{m,n}$, $e^{C,Up}_{m,n}$, $e^{E,Down}_{n,m}$, $e^{C,Down}_{n,m}$ and $g_{m,n}$ are the introduced slack variables. As such, the objective function is now convex.

However, the constraints (30c)–(30g) are non-convex. We define the function $f(u,v) = B\log_2(1+(C_1+\frac{C_2}{u})\frac{\gamma}{v^{a/2}})$. Note that $f(u,v)$ is a convex function of the variables u and v, which means $f(u,v)$ can be approximated by its lower bound at any points that satisfy $\bar{u}>0$ and $\bar{v}>0$. Therefore, we obtain the following inequality [20]

$$\begin{aligned} f(u,v) \ge f\left(\bar{u},\bar{v}\right) &- \frac{\gamma B C_2\left(u-\bar{u}\right)}{\bar{u}\ln 2\left[\bar{u}\left(\bar{v}^{a/2}\right)+\gamma\left(\bar{u}C_1+C_2\right)\right]} \\ &- \frac{\beta\gamma B\left(\bar{u}C_1+C_2\right)\left(v-\bar{v}\right)}{\bar{v}\ln 4\left[\bar{u}\left(\bar{v}^{a/2}\right)+\gamma\left(\bar{u}C_1+C_2\right)\right]}. \end{aligned} \tag{31}$$

Based on (31), we make $u = 1+e^{-(C_3+C_4 g_{m,n})}$, $v = d^2_{m,n}$, $\bar{u} = \bar{u}^{(l)}_{m,n} = 1+e^{-(C_3+C_4 g^{(l)}_{m,n})}$, and $\bar{v} = \bar{v}^{(l)}_{m,n} = \|\mathbf{u}^{(l)}_n - \mathbf{u}_m\|^2 + h^2_n$. As such, the constraint (30c) can be converted to the following convex form

$$e^{E,Up}_{m,n} \le \bar{r}^{E,Up}_{m,n}\left(g_{m,n}, \mathbf{u}_n\right), \tag{32}$$

where

$$\begin{aligned} &\bar{r}^{E,Up}_{m,n}\left(g_{m,n}, \mathbf{u}_n\right) \\ &= r^{E,Up}_{m,n}\left(g^{(l)}_{m,n}, \mathbf{u}^{(l)}_n\right) - b^{up,(l)}_{m,n}\left(e^{-(C_3+C_4 g_{m,n})} - e^{-\left(C_3+C_4 g^{(l)}_{m,n}\right)}\right) \\ &\quad - c^{up,(l)}_{m,n}\left(\|\mathbf{u}_n-\mathbf{u}_m\|^2 - \left\|\mathbf{u}^{(l)}_n-\mathbf{u}_m\right\|^2\right), \end{aligned} \tag{33}$$

with $b^{up,(l)}_{m,n}$ and $c^{up,(l)}_{m,n}$ respectively given by

$$\begin{aligned} b^{up,(l)}_{m,n} &= \frac{\gamma^{up}_{m,n} B_{m,n} C_2}{\bar{u}^{(l)}_{m,n}\ln 2\left[\bar{u}^{(l)}_{m,n}\left(\bar{v}^{(l)}_{m,n}\right)^{\frac{a_1}{2}} + \gamma^{up}_{m,n}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)\right]}, \\ c^{up,(l)}_{m,n} &= \frac{a_1\gamma^{up}_{m,n} B_{m,n}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)}{\bar{v}^{(l)}_{m,n}\ln 4\left[\bar{u}^{(l)}_{m,n}\left(\bar{v}^{(l)}_{m,n}\right)^{\frac{a_1}{2}} + \gamma^{up}_{m,n}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)\right]}. \end{aligned} \tag{34}$$

Similarly, the constraint (30d) can be upper bounded by

$$e^{E,Down}_{n,m} \le \bar{r}^{E,Down}_{n,m}\left(g_{m,n}, \mathbf{u}_n\right), \tag{35}$$

where

$$\begin{aligned} &\bar{r}^{E,Down}_{n,m}\left(g_{m,n}, \mathbf{u}_n\right) \\ &= r^{E,Down}_{n,m}\left(g^{(l)}_{m,n}, \mathbf{u}^{(l)}_n\right) \\ &\quad - b^{down,(l)}_{n,m}\left(e^{-(C_3+C_4 g_{m,n})} - e^{-\left(C_3+C_4 g^{(l)}_{m,n}\right)}\right) \\ &\quad - c^{down,(l)}_{n,m}\left(\|\mathbf{u}_n-\mathbf{u}_m\|^2 - \left\|\mathbf{u}^{(l)}_n-\mathbf{u}_m\right\|^2\right), \end{aligned} \tag{36}$$

with $b^{down,(l)}_{n,m}$ and $c^{down,(l)}_{n,m}$ respectively given by

$$\begin{aligned} b^{down,(l)}_{n,m} &= \frac{\gamma^{down}_{n,m} B_{m,n} C_2}{\bar{u}^{(l)}_{m,n}\ln 2\left[\bar{u}^{(l)}_{m,n}\left(\bar{v}^{(l)}_{m,n}\right)^{\frac{a_1}{2}} + \gamma^{down}_{n,m}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)\right]}, \\ c^{down,(l)}_{n,m} &= \frac{a_1\gamma^{down}_{n,m} B_{m,n}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)}{\bar{v}^{(l)}_{m,n}\ln 4\left[\bar{u}^{(l)}_{m,n}\left(\bar{v}^{(l)}_{m,n}\right)^{\frac{a_1}{2}} + \gamma^{down}_{n,m}\left(\bar{u}^{(l)}_{m,n}C_1 + C_2\right)\right]}. \end{aligned} \tag{37}$$

Now the constraints (30c) and (30d) have been successfully converted into convex constraints (32) and (35), respectively. Next, we deal with the non-convex constraints (30e)–(30f). We firstly define two functions $f(x) = \frac{h}{x^{1/2}}$ and $f(y) = B\log_2(1+\frac{\gamma}{y^{a/2}})$. Note that $f(x)$ and $f(y)$ are convex functions of the variable x and y, respectively, which means they can separately be approximated by their lower bound at any points that satisfy $\bar{x}>0$ and $\bar{y}>0$. Therefore, we can get the following inequalities

$$f(x) \ge f\left(\bar{x}\right) - \frac{h\left(\bar{x}\right)}{2\bar{x}^{3/2}}, \tag{38}$$

$$f(y) \ge f\left(\bar{y}\right) - \frac{\beta\gamma B\left(y-\bar{y}\right)}{\ln 4\cdot\bar{y}\left(\bar{y}^{\beta/2}+\gamma\right)}. \tag{39}$$

Based on (38) and (39), we substitute the functions at right side of the constraints (30e)–(30g) with their lower bounds. Specifically, in constraint (30e), we set $f(x) = \frac{h_n}{d_{m,n}}$ and $\bar{x} = (d^{(l)}_{m,n})^2$. In constraints (30f) and (30g), we set $f_1(y) = R^{C,Up}_{m,n}(\mathbf{u}_n)$ and $f_2(y) = R^{C,Down}_{n,m}(\mathbf{u}_n)$ with the same $\bar{y} = (d^{(l)}_n)^2$, respectively.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Resource allocation algorithm based on SQP.

Input : UAV deployment scheme $\mathbf{u}$, caching and offloading decision schemes $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, error tolerance ε_2 and ε_3, maximum number of iterations I^2_{iter}, I^3_{iter}.
Output: the optimal computation resource and transmit power allocation schemes $\mathbf{F}^*$ and $\mathbf{P}^*$.

Randomly initialize $\mathbf{F}(0)$ and $\mathbf{P}(0)$;
$j = 1$;
repeat
  Solve the computation resource allocation problem (45) by using the CVX tool to obtain $\mathbf{F}(j)$;
  Begins the SQP algorithm:
    Randomly initialize $\mathbf{v}^{(0)}(j)$ and $\lambda_i^{(0)}(j)$ that satisfy $g_i\left(\mathbf{v}^{(0)}(j)\right) \geq 0$ and $\lambda_i^{(0)}(j) \geq 0$;
    $k = 1$;
    repeat
      Compute $H_k(j)$, $C_k(j)$, $A_k(j)$ and $B_k(j)$ based on equation (54);
      Solve the QP problem (53) to get $s^{(k)}(j)$ and compute $\lambda_i^{(k)}(j)$ based on (55) ;
      Set $v_{m,n}^{(k+1)}(j) = v_{m,n}^{(k)}(j) + s^{(k)}(j)$;
      $k = k + 1$;
    until $\left\|s^{(k)}(j)\right\| < \varepsilon_3$ *or* $k > I^3_{iter}$;
  Obtain $\mathbf{P}(j)$ based on the SQP algorithm;
  $j = j + 1$;
until $\left|W^{total}(j) - W^{total}(j-1)\right| < \varepsilon_2$ or $j > I^2_{iter}$;

Therefore, we can get the following convex constraints

$$g_{m,n} \leq \frac{h_n}{d_{m,n}^{(l)}} - \frac{h_n\left(\|\mathbf{u}_n - \mathbf{u}_m\|^2 - \left\|\mathbf{u}_n^{(l)} - \mathbf{u}_m\right\|^2\right)}{2\left(d_{m,n}^{(l)}\right)^{3\,2}}, \tag{40}$$

$$e_{m,n}^{C,Up} \leq R_{m,n}^{C,Up}\left(\mathbf{u}_n^{(l)}\right) - \frac{a_2\gamma_n^{up}\left(\|\mathbf{u}_n - \mathbf{u}_0\|^2 - \left\|\mathbf{u}_n^{(l)} - \mathbf{u}_0\right\|^2\right)}{\ln 4\left(d_n^{(l)}\right)^2\left[\left(d_n^{(l)}\right)^{a_2} + \gamma_n^{up}\right]}, \tag{41}$$

$$e_{n,m}^{C,Down} \leq R_{n,m}^{C,Down}\left(\mathbf{u}_n^{(l)}\right) - \frac{a_2\gamma_n^{dw}\left(\|\mathbf{u}_n - \mathbf{u}_0\|^2 - \left\|\mathbf{u}_n^{(l)} - \mathbf{u}_0\right\|^2\right)}{\ln 4\left(d_n^{(l)}\right)^2\left[\left(d_n^{(l)}\right)^{a_2} + \gamma_n^{dw}\right]}, \tag{42}$$

where $\gamma_n^{up} = \frac{p_{uav}\alpha_0}{\sigma_n^2}$ and $\gamma_n^{dw} = \frac{P_{cloud}\alpha_0}{\sigma_n^2}$.

Based on the above analysis, the EUAV deployment problem can be converted into a convex optimization problem as

$$P1: \min_{\mathbf{u},\mathbf{e},\mathbf{g}} \sum_{m=1}^{M}\left(\delta_{m,n} + \frac{c_1 O_m}{e_{m,n}^{E,Up}} + \frac{c_2 O_m}{e_{m,n}^{C,Up}} + \frac{c_3 C_m}{e_{n,m}^{E,Down}} + \frac{c_4 C_m}{e_{n,m}^{C,Down}}\right)$$
$$\text{s.t.} \quad \text{(25i), (30a), (30b), (32), (35), (40)–(42),} \tag{43}$$

which can be solved by the CVX solver [29]. The deployment algorithm of EUAVs is detailed in Algorithm 1, in which the optimization problem P1 is iteratively solved until the relevant cost converges to a predefined threshold.

B. Resource Allocation Subproblem

Next, we focus on the resource allocation subproblem. With the given caching and offloading decision scheme and the EUAV deployment scheme, the problem in (25) can be expressed as

$$\min_{\mathbf{P},\mathbf{F}} \quad W^{total}$$
$$\text{s.t.} \quad \text{(25e)–(25h).} \tag{44}$$

which is non-convex and difficult to be directly solved. To tackle this problem, we decompose it into a computing resource allocation subproblem and a transmit power allocation subproblem, respectively, given by

$$\min_{\mathbf{F}} \quad W^{total}$$
$$\text{s.t.} \quad \text{(25e), (25f).} \tag{45}$$

$$\min_{\mathbf{P}} \quad W^{total}$$
$$\text{s.t.} \quad \text{(25e)–(25h).} \tag{46}$$

Note that, if the power allocation is determined, (45) is a convex optimization problem with regard to the variables $f_{n,m}$ and $f_{c,m}$, which can be solved by the CVX solver. However, for a given computation resource allocation, (46) is still non-convex. Therefore, we first use the variable transformation method by introducing the variables $\nu_{m,n}^1 = 1/R_{(p_{m,n})}$ and $\nu_{m,n}^2 = 1/R_{(q_{n,m})}$. According to the equations (4) and (5), the relationship between the transmit power and transmission rate can be expressed as

$$p_{m,n} = \left(2^{R_{(p_{m,n})}/B_{m,n}} - 1\right)\kappa,$$
$$q_{n,m} = \left(2^{R_{(q_{n,m})}/B_{m,n}} - 1\right)\kappa, \tag{47}$$

where $\kappa = \kappa_1/\kappa_2$ with $\kappa_1 = \sigma_{m,n}^2 d_{m,n}^{a_1}$ and $\kappa_2 = (C_1 + \frac{C_2}{1+e^{-(C_3+(C_4 H_n/d_{m,n}))}})\alpha_0$. As such, the optimization problem (46) can be reformulated as

$$\min_{\mathbf{p}} \sum_{n=1}^{N}\sum_{m=1}^{M}\left\{K_1\nu_{m,n}^1\left[\omega_{m,1} + \omega_{m,2}\kappa\left(2^{\frac{1}{B_{m,n}\nu_{m,n}^1}} - 1\right)\right] + C_m\nu_{m,n}^2\left[\omega_{m,1} + \omega_{m,2}\kappa\left(2^{\frac{1}{B_{m,n}\nu_{m,n}^2}} - 1\right)\right] + K_2 + K_3\right\}$$
$$\text{s.t. } L_{m,n}^{total}\left(\nu_{m,n}^1, \nu_{m,n}^2\right) \leq T_{\max}, (a)$$
$$E_{m,n}^{total}\left(\nu_{m,n}^1, \nu_{m,n}^2\right) \leq E_{\max}, (b)$$
$$\nu_{m,n}^1 \geq \frac{1}{R_{(p_{m,n}^{\max})}}, (c)$$
$$\nu_{m,n}^2 \geq \frac{1}{R_{(q_{n,m}^{\max})}}, (d) \tag{48}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

where the expressions of the parameters K_1-K_5, $L_{m,n}^{total}$ and $E_{m,n}^{total}$ in (49), shown at the bottom of the page.

Lemma 1: (48) is a convex optimization problem.

Proof: We define the function $f(v) = v \cdot 2^{\frac{1}{B_{m,n} \cdot v}}$, and its first-order derivative with respect to v is given by

$$f'(v) = 2^{\frac{1}{B_{m,n} \cdot v}} \left(1 - \frac{\ln 2}{B_{m,n} \cdot v}\right) - 1. \tag{50}$$

The second-order derivative with respect to v is given by

$$f''(v) = 2^{\frac{1}{B_{m,n} \cdot v}} \frac{(\ln 2)^2}{B_{m,n}^2 v^3}. \tag{51}$$

When $v > 0$, the second-order derivative of $f(v)$ satisfies $f''(v) > 0$, which means $f(v)$ is a convex function with respect to the variable v. As such, the objective function and constraints (48a), (48b) in (48) are the combinations of convex and linear functions with respect to the variables $v_{m,1}$ and $v_{m,2}$. Therefore, (48) is a convex optimization problem. The proof is completed.

Note that both the objective function and constraints in (48) contain non-linear function, which makes it a non-linear programming problem and can be solved by the SQP algorithm. The core idea of the SQP algorithm is to convert the complex nonlinearly constrained optimization problem into a relatively simple quadratic programming (QP) problem. Specifically, we set the objective function in (48) as $f(\nu_{m,n}^1, \nu_{m,n}^2)$, and represent the constraints (48a)–(48d) as $g_1(\nu_{m,n}^1, \nu_{m,n}^2) \geq 0$, $g_2(\nu_{m,n}^1, \nu_{m,n}^2) \geq 0$, $g_3(\nu_{m,n}^1, \nu_{m,n}^2) \geq 0$ and $g_4(\nu_{m,n}^1, \nu_{m,n}^2) \geq 0$, respectively. Then, the Lagrange multipliers $\lambda_1^{(k)}$, $\lambda_2^{(k)}$, $\lambda_3^{(k)}$, and $\lambda_4^{(k)}$ are introduced to obtain the Lagrange function as

$$L\left(\nu_{m,n}^1, \nu_{m,n}^2, \boldsymbol{\lambda}\right) = f\left(\nu_{m,n}^1, \nu_{m,n}^2\right) - \sum_{i=1}^{4} \lambda_i g_i\left(\nu_{m,n}^1, \nu_{m,n}^2\right). \tag{52}$$

Based on (52) and Karush-Kuhn-Tucker (KKT) conditions, we can get the following quadratic programming problem

$$\begin{aligned} P2: \min \quad & \sum_{n=1}^{N} \sum_{m=1}^{M} \frac{1}{2} s^T H_k s + {C_k}^T s \\ \text{s.t.} \quad & A_k s \leq -B_k, \end{aligned} \tag{53}$$

where

$$H_k = \nabla_{\mathbf{v}^{(k)}}^2 f\left(\mathbf{v}^{(k)}\right) - \sum_{i=1}^{4} \lambda_i^{(k)} \nabla_{\mathbf{v}^{(k)}}^2 g_i\left(\mathbf{v}^{(k)}\right),$$

$$C_k = \nabla f_{\mathbf{v}^{(k)}}\left(\mathbf{v}^{(k)}\right),$$

$$A_k = \left[\nabla_{\mathbf{v}^{(k)}} g_1\left(\mathbf{v}^{(k)}\right), \ldots, \nabla_{\mathbf{v}^{(k)}} g_4\left(\mathbf{v}^{(k)}\right)\right]^T,$$

$$B_k = \left[g_1\left(\mathbf{v}^{(k)}\right), \ldots, g_4\left(\mathbf{v}^{(k)}\right)\right]^T, \tag{54}$$

where $\mathbf{v}^{(k)} = [v_{m,1}^{(k)}, v_{m,2}^{(k)}]$, $\lambda_i^{(k)}$ is the Lagrange multiplier of the i-th constraint at the k-th iteration. The update principle of the Lagrange multiplier is given by

$$\lambda_i^{(k+1)} = \left(A_a^{(k)}\left(A_a^{(k)}\right)^T\right)^{-1} A_a^{(k)}\left(H_k s + C_k\right), \tag{55}$$

where $A_a^{(k)}$ represents all the rows in $A^{(k)}$ at the k-th iteration that satisfies the condition $A_k s + B_k = 0$. At each iteration of the SQP algorithm, the descent direction searching problem is converted into a quadratic optimal problem, and the obtained solution is the Newton direction of the Lagrange function with respect to variable v.

The resource allocation algorithm is demonstrated in Algorithm 2. In this algorithm, we solve the optimization problem (45) and (46) iteratively until converging to a prescribed accuracy to obtain the optimal computation and power allocation results $\mathbf{F}$ and $\mathbf{P}$, respectively.

C. Caching and Offloading Decision Subproblem

In this subsection, we solve the caching and offloading decision subproblem. To this end, we re-express the problem in (25) for given EUAV deployment and resource allocation, as follows

$$\begin{aligned} \min_{\boldsymbol{\alpha}, \boldsymbol{\beta}} \quad & W^{total} \\ \text{s.t.} \quad & \text{(25a)–(25d)}. \end{aligned} \tag{56}$$

Note that the problem in (56) is a NP-hard problem, due to the computational complexity is $O(M^{N+2})$ for computation offloading decision determination only, where M and N denote the number of users and EUAVs, respectively.

To address this issue, we propose a modified DQN-based algorithm to perform the caching and offloading decision, where the central controller manages and coordinates the users and UAVs in a unified manner. Note that the DQN method is based on the neural network and Q-learning. In the DQN,

$$K_1 = O_m\left(\beta_{m,n}^E + \beta_{m,n}^C\right),\ K_2 = \delta_{m,n} + \frac{O_m \beta_{m,n}^C \left(\omega_{m,1} + \omega_{m,2} p_{uav}\right)}{R_{m,n}^{C,Up}},\ K_3 = \left(1 - \alpha_{n,m}\right) \frac{C_m \left(\omega_{m,1} + \omega_{m,2} p_{cloud}\right)}{R_{n,m}^{C,Down}},$$

$$K_4 = \left(1 - \beta_{m,n}^E - \beta_{m,n}^C\right) \frac{F_m}{f_m} + \beta_{m,n}^E \frac{F_m}{f_{n,m}} + \beta_{m,n}^C \left(\frac{O_m}{R_{m,n}^{C,Up}} + \frac{F_m}{f_{c,m}}\right) + \left(1 - \alpha_m\right) \frac{C_m}{R_{n,m}^{C,Down}},$$

$$K_5 = \left(1 - \beta_{m,n}^E - \beta_{m,n}^C\right) \varepsilon f_m^2 F_m + \beta_{m,n}^E E_{n,m}^{com} + \beta_{m,n}^C \left(\frac{O_m p_{uav}}{R_{m,n}^{C,Up}} + E_{c,m}^{com}\right) + \left(1 - \alpha_m\right) \frac{C_m p_{cloud}}{R_{n,m}^{C,Down}},$$

$$L_{m,n}^{total} = K_1 \nu_{m,n}^1 + C_m \nu_{m,n}^2 + K_4,\ E_{m,n}^{total} = K_1 \nu_{m,n}^1 \kappa \left(2^{\frac{1}{B_{m,n} \nu_{m,n}^1}} - 1\right) + C_m \nu_{m,n}^2 \kappa \left(2^{\frac{1}{B_{m,n} \nu_{m,n}^2}} - 1\right) + K_5. \tag{49}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

we apply the experience replay and target network technologies to enhance the learning reliability and efficiency. After each iteration, the experience tuple (s_t, a_t, r_t, s_{t+1}) is stored in the experience replay buffer $\mathcal{E}$, where in this paper, the state s_t, action a_t and reward r_t are respectively defined as follows

- State s_t: The state obtained from observing the environment can be represented as $s_t = (W_t^{total}, \mathbf{u}_t, \mathbf{\Psi}_t, \mathbf{\Phi}_t)$, which contains the system cost, deployment of the EUAVs, the remaining caching and computation resources of the EUAVs at current time slot t, respectively. Note that we have $\mathbf{\Phi}_t = (\Phi_1^t, \Phi_2^t, \ldots, \Phi_N^t)$ with $\Phi_n^t = \phi_n - \sum_{m=1}^{M} \phi_{n,m} C_m$, and $\mathbf{\Psi}_t = (\Psi_1^t, \Psi_2^t, \ldots, \Psi_N^t)$ with $\Psi_n^t = \psi_n^t - \sum_{m=1}^{M} \beta_{m,n}^{E,t} f_{n,m}$.
- Action a_t: The action includes the caching and offloading decision, $a_t = (\boldsymbol{\alpha}_m^t, \boldsymbol{\beta}_m^{E,t}, \boldsymbol{\beta}_m^{C,t} | \forall m \in \mathcal{M})$, where $\boldsymbol{\alpha}_m^t = (\alpha_{m,1}^t, \alpha_{m,2}^t, \ldots, \alpha_{m,N}^t)$, $\boldsymbol{\beta}_m^{E,t} = (\beta_{m,1}^{E,t}, \beta_{m,2}^{E,t}, \ldots, \beta_{m,N}^{E,t})$, and $\boldsymbol{\beta}_m^{C,t} = (\beta_{m,1}^{C,t}, \beta_{m,2}^{C,t}, \ldots, \beta_{m,N}^{C,t})$.
- Reward r_t: After executing the action a_t, the system transits from the state s_t to the state s_{t+1} and generates the corresponding reward r_t. We define the utility function $\mathrm{U} = -W^{total}$ to specify the optimization objective, where a higher cost leads to a smaller utility value. However, U cannot be directly used as the reward function since it cannot evaluate the desirability of the action taken under a particular state. To this end, we consider adopting the difference of two consecutive utility values and as such, define the reward function as

$$r_t = \begin{cases} r_1, \text{ if } \mathrm{U_{t+1}} - \mathrm{U_t} > \zeta \\ r_2, \text{ if } \mathrm{U_{t+1}} - \mathrm{U_t} < \zeta \\ 0, \text{ otherwise} \end{cases} \tag{57}$$

  where $r_1 > 0$, $r_2 < 0$, and ζ is a tunable parameter that adjust the agent's sensitivity to changes of utility values. Moreover, we also introduce the penalties to r_t when constraints (25c) and (25d) are violated. Based on the above reward function, the system is able to learn to judge the quality of the action, and thus, obtain the optimal caching and offloading decision policies.

When $\mathcal{E}$ is full, the newest samples will replace the oldest samples to be stored in the buffer. θ is the weight of the neural network and is updated based on the minimum mean square error principle, which is given by

$$L(\theta) = \frac{1}{N_b} \sum_{t=1}^{N_b} \left(y_t - Q(s_t, a_t; \theta)\right)^2, \tag{58}$$

where y_t is the target value, $y_t = r_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}; \theta)$, $\gamma \in (0, 1]$ represents the discount factor for weighting future rewards, and N_b is the size of the mini-batch sampled from replay buffer $\mathcal{E}$. The parameter of the target network is θ_{tar}. We adopt the soft updating principle to update $\theta^{tar} = \theta$ every few episodes.

In order to enhance the performance of sampling and training, we propose a cost sensitive replay memory method, in which the sumtree [30] is introduced to record the training samples based on their weights. The weight of the sample i is given by $\eta_i = \tanh Z / c_i$, where Z is the cost influence coefficient and c_i is the cost of sample i. After calculating all the weights, the sumtree is updated by adding the sample i and its corresponding weight η_i as a new leaf. As such, the samples with the top N_b highest weights will be chosen to train the network.

Algorithm 3: SS-DQN Algorithm for Caching and Offloading Decisions.

Input: episode length L_{ep}, discount factor γ, replay buffer size $\mathcal{E}$, mini-batch size N_b, number of steps to update the target network L_s;
Output: Well-trained neural network parameters;
Randomly initialize the prediction network $Q(s_t, a_t; \theta)$ and target network $Q'(s_t, a_t; \theta^{tar})$ with weights θ and θ_{tar};
$l = 0$;
for episode $l = 0{:}L_{ep}$ **do**
  Randomly initialize the EUAV deployment scheme $\mathbf{u}_0$ and the resource allocation scheme $\mathbf{F}_0$ and $\mathbf{P}_0$. Then observe the environment to obtain the state s_0;
  for time step $t = 0{:}T$ **do**
    Select action: $a_t = \begin{cases} \arg\max_{a_t} Q(s_t, a_t; \theta) \text{ with probability } 1-\epsilon \\ \text{random action with probability } \epsilon \end{cases}$;
    Execute the corresponding action a_t;
    Compute the EUAV deployment scheme $\mathbf{u}_t$ and the resource allocation scheme $\mathbf{F}_t$ and $\mathbf{P}_t$ based on the Algorithm 1 and 2, respectively;
    Compute the reward r_t based on obtained solutions a_t, $\mathbf{u}_t$, $\mathbf{F}_t$ and $\mathbf{P}_t$. Then the system transits to the next state s_{t+1};
    Store the experience tuple (s_t, a_t, r_t, s_{t+1}) into replay buffer $\mathcal{E}$;
    if $\mathcal{E}$ is full **then**
      Randomly sample N_b tuples from $\mathcal{E}$ to form the mini batch;
      for $i = 1{:}N_b$ **do**
        Obtain the target value y_t;
      end for
      Update the weight of evaluation network θ by minimizing $L(\theta)$ based on (58);
    end if
    if Remainder $(\frac{l}{L_s})$==0 **then**
      Update the weight of target network θ_{tar} by $\theta_{tar} = \theta$;
    end if
    $t = t + 1$, $l = l + 1$;
  end for
end for

Finally, based on the proposed SCP-, SQP-, and DQN- based algorithms, we propose the SS-DQN algorithm shown in Algorithm 3. In this algorithm, we adopt a reward incentive mechanism, in which with the EUAV deployment scheme $\mathbf{u}$ and the resource allocation scheme $\mathbf{F}$ and $\mathbf{P}$ obtained by Algorithm 1 and Algorithm 2, the relevant reward can be calculated as an incentive to motivate the system to take actions toward the direction of low-cost. In particular, the SS-DQN algorithm includes

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

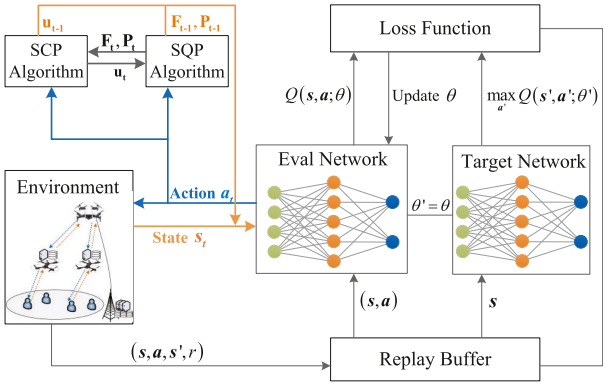


Fig. 2. The framework of the proposed SS-DQN.

two phases, namely, the training phase and the test phase. In the training phase, at each time slot t, the optimal caching and offloading decision policies $\boldsymbol{\alpha}_t$ and $\boldsymbol{\beta}_t$ are computed via DQN with $\mathbf{F}_{t-1}$ and $\mathbf{P}_{t-1}$ obtained by Algorithm 1 and Algorithm 2 at the last time slot $t-1$. The actions $\boldsymbol{\alpha}_t$ and $\boldsymbol{\beta}_t$ are then used to compute $\mathbf{u}_t$, $\mathbf{F}_t$ and $\mathbf{P}_t$ based on Algorithm 1 and Algorithm 2, respectively. As such, the reward r_t is computed and the system transit to the next state s_{t+1}. The above processes are performed iteratively in each time slot until the training phase is over. In the test phase, the well-trained neural network is employed to perceive the environment and react in real-time. The framework of the proposed SS-DQN algorithm is illustrated in Fig. 2.

Note that with the help of SCP and SQP algorithms, SS-DQN algorithm only needs to train the caching and offloading decision variables instead of all the involved variables in the optimization problem (25), which drastically reduces the corresponding state-action space and effectively enhance the executive efficiency of the DQN algorithm. Since the computational complexity of the SCP and SQP algorithms is polynomial, the overall computational complexity of the proposed SS-DQN algorithm is within an acceptable range.

IV. Simulation Results

In this section, we provide numerical results to validate the performance of the proposed algorithm. Specifically, 10 users are uniformly and randomly distributed in a square area of 1000 m $\times$ 1000 m, where three EUAVs and one TUAV are deployed. The 2-D coordinate[2] of the TUAV is fixed as [1000,1000] m. The heights of UAVs and TUAV are set as $h_n = 100$ m and $h_0 = 150$ m, respectively. Following [23], the parameters in (8) are separately set as $C_1 = 0.01$, $C_2 = 0.99$, $C_3 = -4.7$ and $C_4 = 8.9$, which guarantees the maximum possible time-out probability $\delta = 0.0025$. $\alpha_0 = 10^{-5}$. The parameters in the energy consumption model are separately set as $\varepsilon = 10^{-28}$[27], $e_1 = 0.5$, $e_2 = 0.4$, $e_3 = 0.6$, $e_4 = 0.6$ [28]. We set the default values of weights in (23) as $\omega_{m,1} = \omega_{m,2} = 0.5$. As such, the latency and energy consumption can have equal importance on the cost evaluation. Other parameters utilized throughout the whole simulation are summarized in Table II, unless otherwise specified.

TABLE II
Simulation Parameters

Allocated bandwidth	10 MHz
Maximum transmit power of users	0.2 W
Maximum transmit power of EUAVs	0.5 W
Transmit power of TUAV	1 W
Path loss exponent	3
Noise power spectral density	-174 dBm/Hz
Error tolerance $\varepsilon_1, \varepsilon_2$ and ε_3	0.01
Computation capacity of EUAV	4 GHz
Computation capacity of the cloud	10 GHz
CPU frequency of users	1 GHz [32]
Caching capacity of EUAVs	10 Mb
Size of the offloaded task	$[0.5, 4]$ Mbits
Size of the required cache file	$[0.5, 4]$ Mbits
Required CPU cycles	$[500, 5000] \times 10^6$
Learning rate	0.0001
Discount factor	0.999
Experience replay buffer size	10000
Batch size	128
Soft update rate	0.002
Greedy coefficient	0-0.9

In order to evaluate the performance of our proposed algorithm, we introduce two benchmark strategies, namely, 1) the UAV-enabled local-edge combined (ULEC) scheme, which reserves the local and edge computing parts, and have no cloud center. In this scheme, three EUAVs work as the edge servers as in our proposed algorithm; 2) the UAV and access point (AP)-assisted local-edge combined (UAPLEC) scheme proposed in [32], in which a novel MEC system is considered with two ground MEC servers at the APs and one MEC server carried by the UAV. The locations of the APs are fixed at the edge of the network, while the UAV can fly around to provide faster response to the user requests. Note that the flight energy consumption in the UAPLEC scheme is much larger than the communication and computation energy consumption. Therefore, in this section, we only focus on the energy consumption produced by the communication and computation. In addition, since the users start the computation only after obtaining their required cache files, and the cache capacity of edge servers in the above two comparison schemes is relatively limited. Thus, the computing tasks cannot be executed if their requested files have not been cached in EUAVs or APs, and an extra cost is introduced as a punishment.

In Fig. 3(a), we plot the system cost versus the number of users for different schemes. It can be seen that our proposed scheme induces the smallest cost compared to the two comparison schemes. The powerful processing capabilities of the cloud bring a significant reduction to cost, showing the superiority of the cloud-edge architecture and how a proper offloading decision plays a crucial role in enhancing the system performance. Moreover, with an increasing number of users, the total system cost increases as well. In addition, the performance gap between the ULEC scheme and the UAPLEC scheme is

[2]Note that the 2-D coordinate of the TUAV can be any point within the simulation area, and the trend observed from the figures will not be changed. In our work, considering the limited length of the tether [36] and the complex topographic conditions [37], we choose to place the TUAV at the edge of the simulation area.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

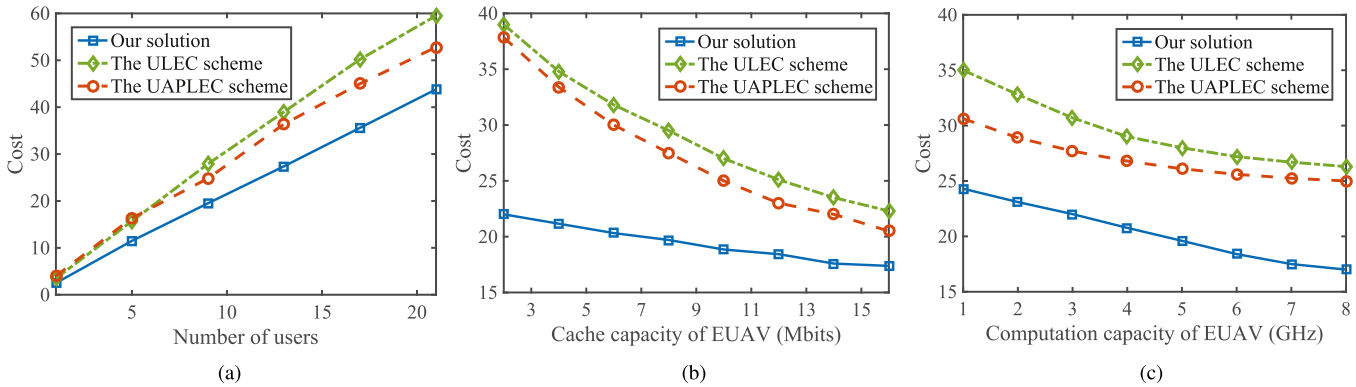


Fig. 3. System cost performance of the considered three schemes at varying number of users (M), cache capacity of each EUAV (ϕ_n) and computation capacity of each EUAV (ψ_n). (a) Cost versus number of users. (b) Cost versus cache capacity of EUAV. (c) Cost versus computation capacity of EUAV.

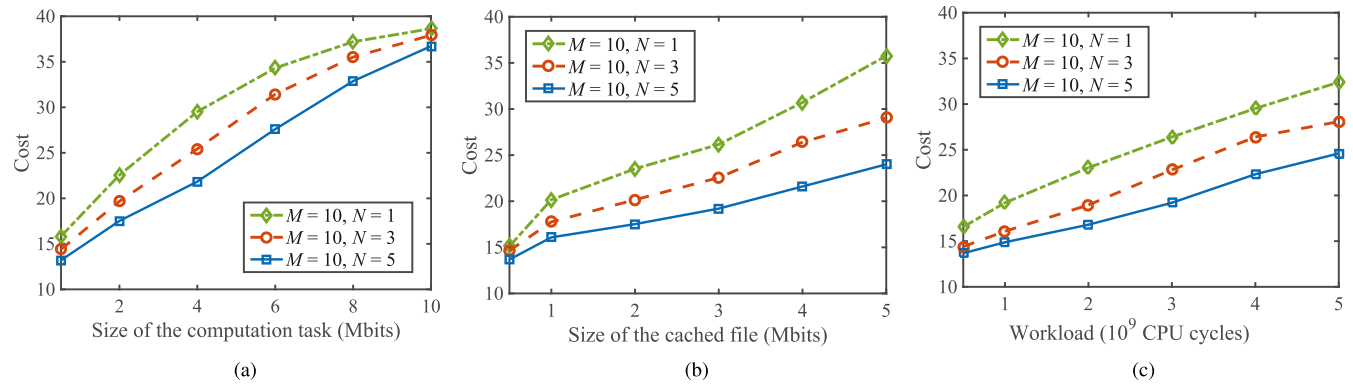


Fig. 4. The impact of parameter settings (including the size of computation tasks, cached files and workloads) on the system cost under the scenario of $M = 10$, $N = \{1, 3, 5\}$. (a) System cost versus size of the computation task. (b) System cost versus size of the cached file. (c) System cost versus the workload.

relatively small, verifying the effectiveness of the reasonable EUAV deployment.

We further examine the effect of cache capacity ϕ_n on the cost performance in Fig. 3(b). It is again observed that our proposed scheme outperforms the other two benchmark solutions. We can also see that the system costs of the three considered schemes decreases with the increase of ϕ_n, due to the enhanced capabilities of the edge to cache more files and provide faster response to the user requests. It is worth noting that the performance gap between our solution and the two comparison schemes is becoming smaller with higher ϕ_n. The reason lies in ϕ_n, when ϕ_n is small, most of users' tasks in the ULEC and UAPLEC schemes cannot be accomplished, resulting in a heavy penalty on the system cost. However, when ϕ_n is relatively large, all users' tasks can be executed normally, and further increasing the capacity is no longer available for enhancing the cost performance, and thus the cost approaches to a certain value.

The effect of computation capacity ψ_n on the cost performance is investigated in Fig. 3(c). Similar to the observations in Fig. 3(b), the increase of ψ_n can bring reduction to the system cost. This is because the enhanced capabilities of the edge make the task computation faster. Moreover, we can observe how the costs of the three considered schemes gradually approach to certain values. This is because the reduction of latency brought by higher computation frequency is offset by the increase of energy consumption. In addition, the performance gain of the ULEC and UAPLEC schemes is smaller than that of our proposed scheme, leading to that the performance gap between our scheme and the two comparison schemes becomes increasingly larger with ψ_n. This observation is different from that in Fig. 3(b). Note that the computing tasks of the users are executed only when the required cache files are obtained, which explains why the limited cache capacity of the two comparison schemes restricts the effect of increasing ψ_n.

The impacts of the parameter settings (i.e., size of the computation task, size of the cached file and the workload) on the system cost are contemplated in Fig. 4. We consider $M = 10$ and $N = \{1, 3, 5\}$. It can be observed from Fig. 4(a) that, with the fixed M and N, the increasing size of computation tasks O_m degrades the cost performance of the system, due to the higher transmission latency of communication links between the users and the EUAVs, the EUAVs and the TUAV. As such, the gain of processing latency produced by powerful computing capabilities of the edge and the cloud will finally be offset by the larger transmission latency, leading to the scenario that all the users choose the local computing mode. In the same setting, we then consider the effect of cached file size C_m and workload F_m in Fig. 4(b) and (c), respectively. It can be seen from Fig. 4(b)

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

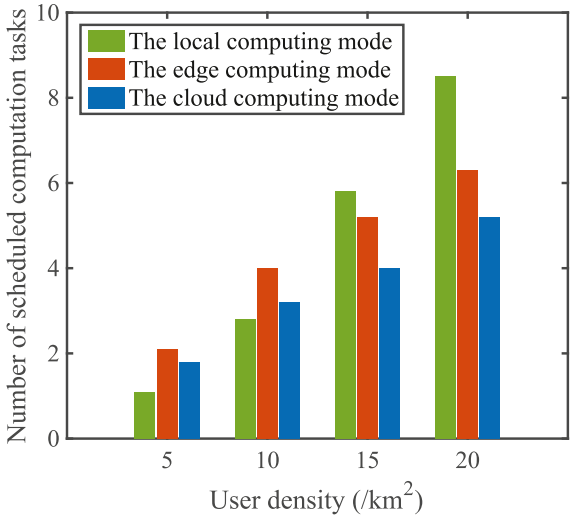


Fig. 5. The selection of computing mode for different user densities.

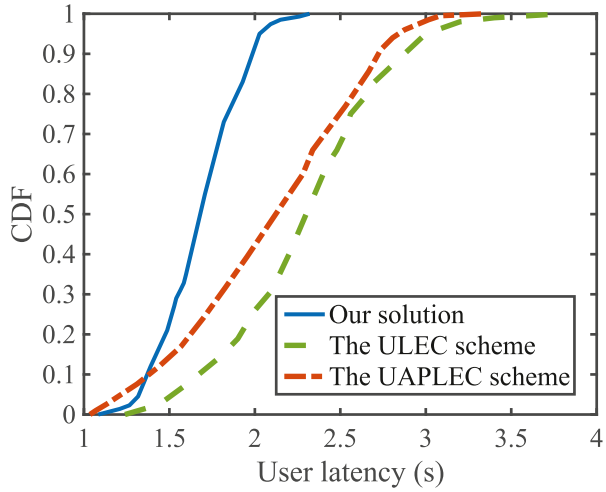


Fig. 6. CDF of users' latency for completing the related computation tasks.

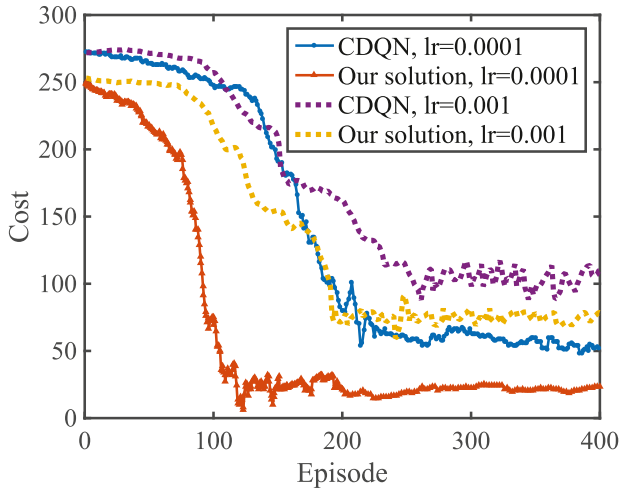


Fig. 7. System cost versus the training episode with various learning rate lr = 0.001 and lr = 0.0001.

that the cost increases with C_m, which is because that a small C_m can be handled by EUAVs easily, while a relatively large C_m requires the involvement of the cloud, leading to the increase of the system cost. Similarly, it can be seen that in Fig. 4(c), with an increasing size of workload F_m, the system cost increases as well. The reason is that when F_m is small, the QoS requirements of users can be met through local computing, which saves the transmission latency to the edge and the cloud. However, when F_m is relatively large, the local computing mode is no longer applicable for satisfying the QoS requirements of the users, which requires the assistance of the more powerful EUAVs and cloud center. In addition, from Fig. 4(a) to (c), we can commonly see that the increase of N simultaneously raises the capabilities of data transmitting, caching and computing, thus, lowering the system cost.

Fig. 5 shows the users' computing mode selection for different user densities. It can be observed that the number of scheduled computation tasks of all three modes increases with the user density. Moreover, as the user density increases, an increased number of users tend to choose the local computing mode. The reason is that the computation resources at the edge and cloud gradually become insufficient for users' demands. We can also observe that the number of users choosing edge computing is always larger than that choosing cloud computing, due to the closer distance between users and EUAVs.

To further analyze the variability of user performance, in Fig. 6, we plot the cumulative distribution function (CDF) of user latency for completing the computation tasks. It can be observed that our proposed scheme can provide the lowest latency on average. Moreover, our proposed scheme is more compact compared with the other two schemes, which means a smaller variance, namely, most of the users in our scheme can complete their computation tasks with a close-to-average time. Such a small variance is extremely significant toward implementing the real-time applications, which require low and constant completion time.

Then, we investigate the convergence performance in Fig. 7 with different learning rate lr = 0.001 and lr = 0.0001. We also introduce a complete DQN (CDQN) algorithm [33] as a comparison. Specifically, the CDQN scheme is a pure DQN algorithm, where both the caching and offloading decisions and the resource allocation strategies are jointly optimized by the centralized DQN controller. Note that the action space for UAV deployment is too large to be directly solved by DQN, therefore, we assume that the location of the UAVs are determined in advance. It can be observed that when lr = 0.0001, our proposed scheme is stabilized at around 100 episodes. However, not only does the CDQN scheme converges slower, but it also suffers a large performance loss compared to our scheme, even after having reached its convergence. The reason is that the quantization of the transmit power and the computing capacity reduces the chances of obtaining the optimal solutions, and thus, deteriorates its performance. Such an observation shows that our proposed DQN-based scheme that combining advantages of both the DQN and convex optimization methods is an efficient and valuable solution to enhance the system performance. Moreover, we can see that a smaller learning rate contributes to a higher converge

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

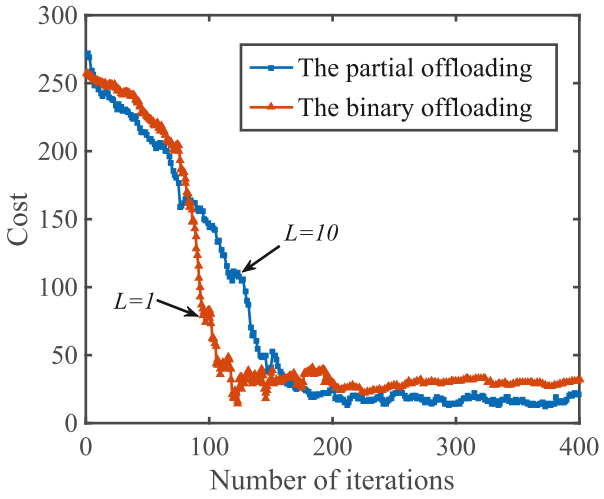


Fig. 8. The performance comparison between the binary and partial offloading.

speed, a smaller jitter, and a better cost performance, which motivate our choice to set the default value of lr as 0.0001.

Fig. 8 shows how the partial offloading mode can enhance the cost performance. To this end, we compare the performance of the binary offloading mode to that of the partial offloading mode with $L = 10$, where each computation task is assumed to be further divided into 10 equal-size segments, as such, the original 10 computation tasks with size [0.5,4] Mbits in the binary offloading scenario become 100 tasks with size [50,400] Kbits. It can be observed in Fig. 7 that the cost performance of the partial offloading mode is better than that of the binary offloading, due to the more sufficient utilization of the local, edge and cloud resources. Moreover, the convergence speed of the partial offloading is slower than that of the binary offloading, due to the enlarged action space.

V. Conclusion

In this paper, we considered a UAV-assisted cloud-edge system, where the EUAVs serve as MEC servers to cooperate with the cloud center to provide cache and computing services for the ground users. The task scheduling between the users, the EUAVs and the cloud was investigated. In order to minimize the weighted sum cost of latency and energy consumption, the caching and offloading decisions, EUAV deployment, as well as transmit power and computation capacity allocation schemes were jointly optimized under the constraints of the EUAV capacities and user-specific latency and energy consumption limits. In order to deal with the NP-hard problem, an novel SS-DQN algorithm was proposed, which combines the advantages of SCP, SQP and DQN methods to deal with the decomposed EUAV deployment problem, the resource allocation problem, as well as the caching and offloading decision problem, respectively. The proposed algorithm allowed the system to learn the optimal parameter tuple that minimize the system cost in a more efficient way. Numerical results showed significant performance gain compared to the existing edge computing only schemes and illustrated the advantages of combining DRL and convex optimization to improve the cost performance and the training efficiency.

References

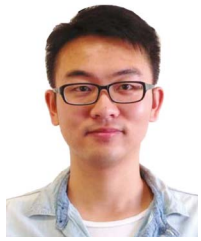
[1] Cisco, "Cisco visual networking index: Global mobile data traffic forecast update, 2018-2023," White Paper, Feb. 2020.
[2] F. Spinelli and V. Mancuso, "Toward enabled industrial verticals in 5G: A survey on MEC-based approaches to provisioning and flexibility," *IEEE Commun. Surv. Tut.*, vol. 23, no. 1, pp. 596–630, Jan.–Mar. 2021.
[3] B. Liu, C. Liu, and M. Peng, "Resource allocation for energy-efficient MEC in NOMA-enabled massive IoT networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 4, pp. 1015–1027, Apr. 2021.
[4] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "AI driven heterogeneous MEC system with UAV assistance for dynamic environment: Challenges and solutions," *IEEE Netw.*, vol. 35, no. 1, pp. 400–408, Jan./Feb. 2021.
[5] S. Gangakhedkar, H. Cao, A. R. Ali, K. Ganesan, M. Gharba, and J. Eichinger, "Use cases, requirements and challenges of 5G communication for industrial automation," in *Proc. IEEE Int. Conf. Commun. Workshops*, Kansas City, MO, USA, 2018, pp. 1–6.
[6] J. Wang, C. Jin, Q. Tang, N. N. Xiong, and G. Srivastava, "Intelligent ubiquitous network accessibility for wireless-powered MEC in UAV-assisted B5G," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 2801–2813, Oct.–Dec. 2021.
[7] L. Wang and M. Hua, "Optimal bit allocation for UAV-enabled mobile communication," in *Proc. 3rd IEEE Int. Conf. Comput. Commun.*, Chengdu, China, 2017, pp. 474–478.
[8] G. Lee, W. Saad, and M. Bennis, "Online optimization for UAV-assisted distributed fog computing in smart factories of industry 4.0," in *Proc. IEEE Glob. Commun. Conf.*, Abu Dhabi, UAE, 2018, pp. 1–6.
[9] L. Pacheco et al., "Towards the future of edge computing in the sky: Outlook and future directions," in *Proc. 2021 17th Int. Conf. Distrib. Comput. Sens. Syst.*, 2021, pp. 220–227.
[10] Z. Yu, Y. Gong, S. Gong, and Y. Guo, "Joint task offloading and resource allocation in UAV-enabled mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3147–3159, Apr. 2020.
[11] A. Sacco, F. Esposito, G. Marchetto, and P. Montuschi, "Sustainable task offloading in UAV networks via multi-agent reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 5, pp. 5003–5015, May 2021.
[12] Z. Ning et al., "5G-enabled UAV-to-community offloading: Joint trajectory design and task scheduling," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3306–3320, Nov. 2021.
[13] E. E. Haber, H. A. Alameddine, C. Assi, and S. Sharafeddine, "UAV-aided ultra-reliable low-latency computation offloading in future IoT networks," *IEEE Trans. Commun.*, vol. 69, no. 10, pp. 6838–6851, Oct. 2021.
[14] G. Faraci, C. Grasso, and G. Schembra, "Design of a 5G network slice extension with MEC UAVs managed with reinforcement learning," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2356–2371, Oct. 2020.
[15] Z. Yuan, J. Jin, L. Sun, K. -W. Chin, and G. -M. Muntean, "Ultra-reliable IoT communications with UAVs: A swarm use case," *IEEE Commun. Mag.*, vol. 56, no. 12, pp. 90–96, Dec. 2018.
[16] Y. Huang, Q. Wu, R. Lu, X. Peng, and R. Zhang, "Massive MIMO for cellular-connected UAV: Challenges and promising solutions," *IEEE Commun. Mag.*, vol. 59, no. 2, pp. 84–90, Feb. 2021.
[17] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Commun. Mag.*, vol. 54, no. 5, pp. 36–42, May 2016.
[18] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.
[19] H. Mei, K. Yang, Q. Liu, and K. Wang, "Joint trajectory-resource optimization in UAV-enabled edge-cloud system with virtualized mobile clone," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5906–5921, Jul. 2020.
[20] A. A. Nasir, "Latency optimization of UAV-enabled MEC system for virtual reality applications under rician fading channels," *IEEE Commun. Lett.*, vol. 10, no. 8, pp. 1633–1637, Aug. 2021.
[21] Y. Liu, K. Xiong, Y. Lu, Q. Ni, P. Fan, and K. B. Letaief, "UAV-aided wireless power transfer and data collection in rician fading," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3097–3113, Oct. 2021.
[22] C. You and R. Zhang, "3D trajectory optimization in rician fading for UAV-enabled data harvesting," *IEEE Trans. Wireless Commun.*, vol. 18, no. 6, pp. 3192–3207, Jun. 2019.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.

[23] S. Sun, G. Zhang, H. Mei, K. Wang, and K. Yang, "Optimizing multi-UAV deployment in 3-D space to minimize task completion time in UAV-enabled mobile edge computing systems," *IEEE Commun. Lett.*, vol. 25, no. 2, pp. 579–583, Feb. 2021.
[24] H. Wei, H. Luo, Y. Sun, and M. S. Obaidat, "Cache-aware computation offloading in IoT systems," *IEEE Syst. J.*, vol. 14, no. 1, pp. 61–72, Mar. 2020.
[25] X. Huang, X. Yang, Q. Chen, and J. Zhang, "Task offloading optimization for UAV-assisted fog-enabled Internet of Things networks," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1082–1094, Jan. 2022.
[26] S. Guo, B. Xiao, Y. Yang, and Y. Yang, "Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing," in *IEEE Conf. Comput. Commun.*, 2016, pp. 1–9.
[27] X. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
[28] L. Rao, X. Liu, M. D. Ilic, and J. Liu, "Distributed coordination of internet data centers under multiregional electricity markets," *Proc. IEEE*, vol. 100, no. 1, pp. 269–282, Jan. 2012.
[29] S. Boyd, S. P. Boyd, and V. Lieven. *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
[30] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–21.
[31] C.-B. Park, B.-S. Park, H.-J. Uhm, H. Choi, and H.-S. Kim, "IEEE 802.15.4 based service configuration mechanism for smartphone," *IEEE Trans. Consum. Electr.*, vol. 56, no. 3, pp. 2004–2010, Aug. 2010.
[32] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "UAV-assisted MEC networks with aerial and ground cooperation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 12, pp. 7712–7727, Dec. 2021.
[33] Y. -C. Wu, T. Q. Dinh, Y. Fu, C. Lin, and T. Q. S. Quek, "A hybrid DQN and optimization approach for strategy and resource allocation in MEC networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4282–4295, Jul. 2021.
[34] X. Hu, K. Wong, K. Yang, and Z. Zheng, "UAV-assisted relaying and edge computing: Scheduling and trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 18, no. 10, pp. 4738–4752, Oct. 2019.
[35] M. Kishk, A. Bader, and M. -S. Alouini, "Aerial base station deployment in 6G cellular networks using tethered drones: The mobility and endurance tradeoff," *IEEE Veh. Technol. Mag.*, vol. 15, no. 4, pp. 103–111, Dec. 2020.
[36] M. A. Kishk, A. Bader, and M.-S. Alouini, "On the 3-D placement of airborne base stations using tethered UAVs," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 5202–5215, Aug. 2020.
[37] S. Zhang, W. Liu, and N. Ansari, "On tethered UAV-assisted heterogeneous network," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 975–983, Jan. 2022.
[38] A. Lapidoth, *A Foundation in Digital Communication*. Cambridge, U.K.: Cambridge Univ. Press, 2017, pp. 500–501.
[39] R. G. Gallager, "Circularly-symmetric Gaussian random vectors," Preprint, pp. 1–9, 2008.
[40] M. Mozaffari, W. Saad, M. Bennis, and M. Debbah, "Performance optimization for UAV-enabled wireless communications under flight time constraints," in *Proc. IEEE Glob. Commun. Conf.*, Singapore, 2017, pp. 1–6.
[41] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7808–7822, Aug. 2020.
[42] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
[43] H. -T. Ye, X. Kang, J. Joung, and Y. -C. Liang, "Optimization for wireless-powered IoT networks enabled by an energy-limited UAV under practical energy consumption model," *IEEE Commun. Lett.*, vol. 10, no. 3, pp. 567–571, Mar. 2021.



Binghong Liu received the B.E. degree in information and communication engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2017. She is currently working toward the Ph.D. degree with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. Her research interests include resource allocation in radio access networks, UAV, NOMA, convex optimization, and deep reinforcement learning.



Chenxi Liu (Senior Member, IEEE) received the B.E. degree from Central South University, Changsha, China, in 2010, and the Ph.D. degree from the University of New South Wales, Sydney, NSW, Australia, in 2016. From 2017 to 2019, he was a Postdoctoral Research Fellow with the Singapore University of Technology and Design, Singapore. Since 2019, he has been with the Beijing University of Posts and Telecommunications, Beijing, China, where he is currently an Associate Professor. His research interests include wireless security, unmanned aerial vehicle-enabled wireless networks, and network intelligence. He was the recipient the Best Paper Award from the IEEE ICC 2022. He is currently the Editor of the IEEE WIRELESS COMMUNICATIONS LETTERS.



Mugen Peng (Fellow, IEEE) received the Ph.D. degree in communication and information systems from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2005. Afterward, he joined BUPT, where he has been a Full Professor with the School of Information and Communication Engineering since 2012. In 2014, he was an Academic Visiting Fellow with Princeton University, Princeton, NJ, USA. He leads a Research Group focusing on wireless transmission and networking technologies with the State Key Laboratory of Networking and Switching Technology, BUPT. He has authored or coauthored more than 100 refereed IEEE journal papers and more than 300 conference proceeding papers. He was the recipient of the 2018 Heinrich Hertz Prize Paper Award, the 2014 IEEE ComSoc AP Outstanding Young Researcher Award, and the Best Paper Award in the JCN 2016 and IEEE WCNC 2015. He is on the Editorial/Associate Editorial Board of the *IEEE Communications Magazine*, the IEEE INTERNET OF THINGS JOURNAL, and IEEE ACCESS.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:00:00 UTC from IEEE Xplore. Restrictions apply.